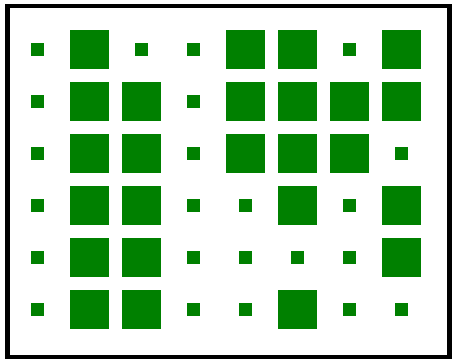




Class 7
Regular Expressions

CS 330
Programming Languages
And Compilers

Vibe coding may be the software development zeitgeist du jour, but vibing with AI isn't all it's cracked up to be. Here are nine ways overly optimistic vibe coders can fail.



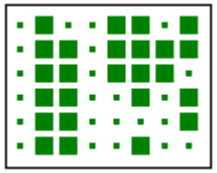
Credit: Oksatox / Shutterstock

Is vibe coding really as easy as they say? Consider the butler, the meat-space equivalent of an AI. There are schools that specialize in teaching new butlers skills like how to serve breakfast or make a perfect martini. But did you know these same schools have a parallel track teaching rich people how to get along with their butler? That's right, rich people *learn* how to hold the teacup correctly, so the butler can gracefully fill it with tea. They even learn what kinds of requests are appropriate and which are not. It's the sort of thing that can't be taught in a two-minute TikTok video.

Being waited on hand and foot isn't easy. There's a right way to hold the teacup, and it's important to know the difference between appropriate requests and ones that are doomed to fail.

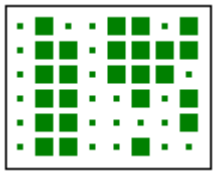
The same goes for **vibe coding** [https://www.infoworld.com/article/3853805/vibe-coding-with-claude-code.html]. Oh, sure, it's amazing what **artificial intelligence** [https://www.infoworld.com/a

LLM Roundtable

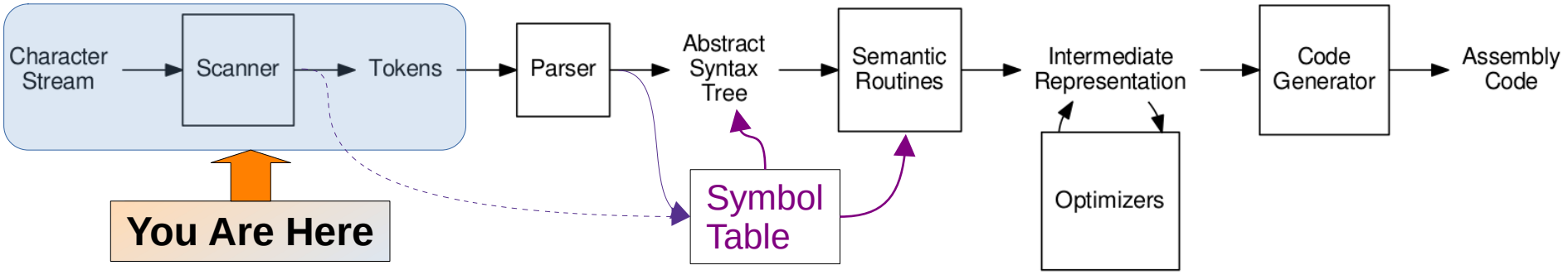


Lexical Analysis Refresher

Compilation Process Overview

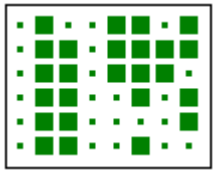


Thain, D. (2020). Introduction to compilers and language design.



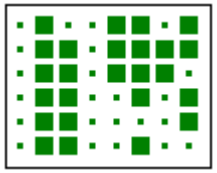
- **Scanner (Lexical analyzer) – tokenizes**
- Parser (Syntax analyzer) – lexical units syntax check
  - Creates symbol table and AST
- Semantic Routines – static semantics analysis/check
- Intermediate Rep – lower level language (optional)
- Optimizer – performance improvements (optional)
- Code Generator – outputs assembly lang (in our case)

Lexical Analysis Overview

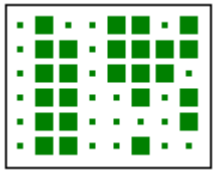


- Lexical analyzer matches patterns in the source code
  - a.k.a. scanner
- **Patterns** are defined to **match** the language's **lexemes**
- **Patterns** are **associated** with **tokens**
- When a lexeme is matched by a pattern, the lexical analyzer associates it with the pattern's token

Implementing a Lexical Analyzer

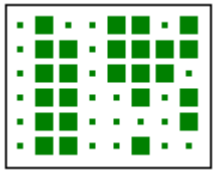


- A couple of ways to implement
  - Describe the definitions of token patterns using a formal language, such as regular expressions (discussed later) and write a program to match those
  - Create a state transition diagram for each token pattern and write a program which implements the diagram



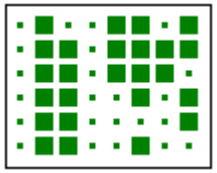
Regular Expressions Background

Common Text-based Interactions



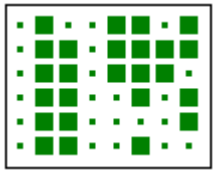
- Searching: Find lines in a file that contain a value or pattern of characters
  - Where is that program file?
  - Where is the web page I designed?
  - Where is the paper I wrote?
- Parsing: Figure out what values a file contains
  - Is the syntax used in this file correct?
  - What language is the text in this file written in?

Regular Expressions: Search Language



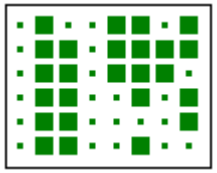
- What are regular expressions?
  - A specialized syntax used to describe patterns of text
  - May be used to find text within an input source (file, network data stream…)
- Why study regular expressions?
  - Used in a variety of tools
    - Editors, file search utilities, compilers, programming language libraries
  - Quick way to search and parse data

Regex: Variations Abound



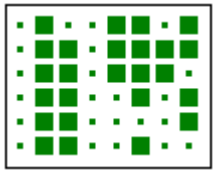
- No standards body defining the syntax
- Over time variations in the syntax have arisen
  - Different implementations will work with different regex syntax
  - Can cause some frustration when working with a new regex environment
- Most of the core regex syntax is common
  - A quick Google search will often help find the alternate syntax for a specific implementation

POSIX Regular Expressions

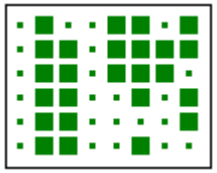


- The POSIX library ships with most Linux distributions
  - Rich set of regex functionality
  - The regex library is available as an API and easily leveraged from C
  - Other languages include their own regex support
    - e.g. Java, Python, Perl
- Terminal based tools such as grep, awk, and vi include support for regular expressions

Regex Engine

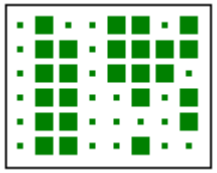


- RegEx is interpreted by a program which compares input text to a pattern, looking for a match
  - The program is a **regex engine**
  - Similar to the lexical analyzer concept
    - The regex matches a lexeme
- The regex engine's input is the regex and text, output is *metadata* identifying the match
  - Where match starts, length of the match



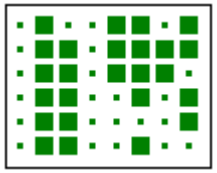
Regular Expression Setup

Process of Using a Regex



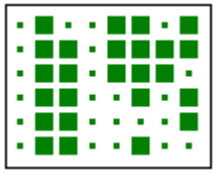
- We need three things
  - The regex, defining the pattern
  - Text to search
  - The regex engine
- Our goal is to determine whether the text contains a match for the pattern
  - If so, where is it in the file

The grep Command Line Search Tool



- Common syntax for using a regex in an editor was to place the pattern between two slashes
  - Before the first slash the user could include the letter '**g**' meaning that the search should be global (search the entire file)
  - After the second slash the user could include the letter '**p**' meaning that each match should be printed on the console
  - This led to the pattern **g*/regular_expression/*p**
- This was used as the name for a command line text searching tool: `grep`

Setup To Work With Regex

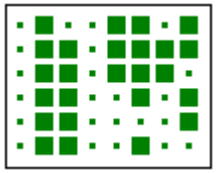


- We'll use a simple program to explore RegEx
  - Useful for testing your regular expressions as you build your lexical analyzer
- Open a terminal, access your `cs330-IndividualLabWork` directory and use the command:

```
git pull
```

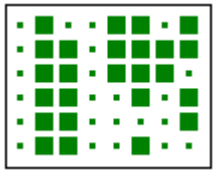
- Should pull a new directory: `regexExploration`
- Change directory into `regexExploration`

Setup To Work With Regex



- The **`regex.c`** program accepts a regex on the command line and searches the “hackers” text file for matching lines, printing those that match
  - If a second string is included it will search that string for a regex match, instead of the file
    - This provides an easy way to test if a RegEx pattern will match specific text, such as when designing RegEx patterns for our B-Minor lexemes

Test the Regex Program



- Compile the program

```
cc regex.c
```

- Test the program with the command:

```
./a.out "wo" "hello, world"
```

- It should output

```
1:hello, world (7,9)
```

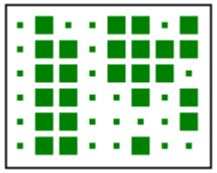
(with the letters “wo” in “world” highlighted)

- Test the program with the command:

```
./a.out "Copyright"
```

- It should output

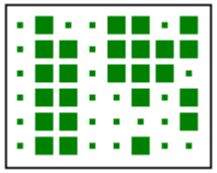
```
1885:what you can do with this work.  Copyright laws in most countries are in (33,42)
```



Important Conceptual Issue!

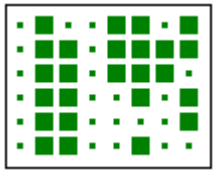
The lexical analyzer (scanner) reads the source code and emits the tokens in the order they are found – it DOES NOT SEARCH for tokens within the source file

As we study regex we'll use text matches to learn how the regex language works, *not how a lexical analyzer works*



Regular Expression Usage

Basic Regular Expression: Characters



- The most basic form of regular expression is a text pattern to match
- Let's find every occurrence of "The"

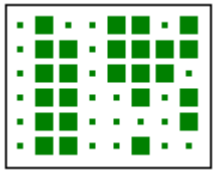
```
./a.out "The"
```

- The output will show each line in the file containing the pattern "The"

    The, There, They…

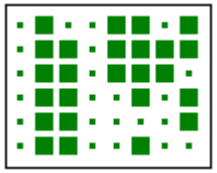
- Note that the highlighted portion of the text is the part that matched the regex

What is the Engine Doing?



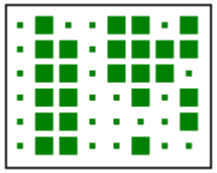
- Engine checks the first character of the text to see if it matches the first ***atom*** of the regex
  - If they match, the process moves on to check the next character in the text against the next atom in the regex. If all the characters/atoms match, in order, a regex match has been found
  - Anytime the match fails, the engine returns to the character after the initial match and restarts the process, back at the first atom

Regex Engine Example (1/2)



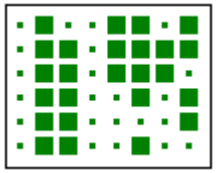
- Given the regex *"The"* and the input text *"Thick darkness came. Their noises scared me."*
- Initially the *Th* of *Thick* matches the *Th* atoms of the regex. However, the *i* in *Thick* doesn't match the third atom, *e*, of the regex
  - The engine starts the regex over, looking at the *h* in *Thick* compared with the first atom, *T*, of the regex
    - These don't match and the process proceeds

Regex Engine Example (2/2)



- Given the regex *"The"* and the input text *"Thick darkness came. Their noises scared me."*
- Eventually the *T* in *Their* matches the first atom and the two following letters in the text match the other regex atoms
- Key takeaway is that we are defining an expected order of atoms to match
  - The utility comes from the regex syntax providing meta-characters, quantifiers, to represent multiple characters for an atom (i.e. at a given position in the text)

Character Sets (Character Classes)



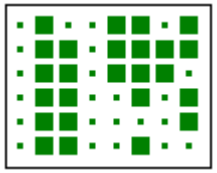
- Previous search found “The” and not “the”
- One way to match both is to use a **character set** (a.k.a. character class)
  - Denoted using square brackets

    `[Tt]` matches a character if it is a T or t

    `[Tt]`he matches “The” and “the”
- A character set defines a group of characters ***any one of which*** will match that atom
- Try it:

```
./a.out "[Tt]he"
```

Character Sets (cont'd)



- May combine any characters to match

  `[AaEeIiOoUu]` matches all vowels

  `[02468]` matches even digits

- Use common ranges in a set

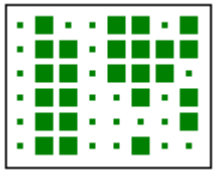
  `[a-zA-Z]` matches all letters

  `[0-9]` matches all digits

- The carat (^) may be used to negate the characters in a character set

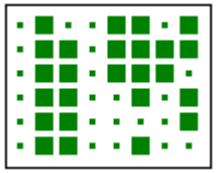
  `[^0-9]` matches any character not a digit

Character Sets (cont'd)



```
./a.out "[A-Z][a-z]e"
./a.out "[0-9][0-9][02468]"
./a.out "[A-Za-z][AaEeIiOoUu][A-Za-z]"
./a.out "[A-Za-z][^AaEeIiOoUu][A-Za-z]"
```

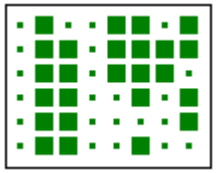
Special Characters



- If we want to use “special characters” as literals in our regular expressions we must escape them with a backslash: `\`
  - Special characters in a regex include `+`, `*`, `|`, `(`, `)`, `^`, `[`, `$`, `{`, and the dot `(.)`
- For example to search for “them.” we’d use the following regex: `them\.`
- Try with and without escaping the dot

```
./a.out "them\."
./a.out "them."
```

Our First Metacharacter: The Dot .

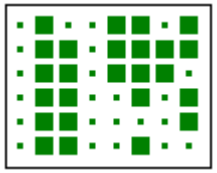


- Dot represents any character for that atom, except a newline
  - Wildcard for a character match

    `[Tt].` matches Ta, ta, Tb, tb, and so on

    `[Tt]..` matches Taa, taa, Tab, and so on
- There must be a character in the text to match (won't match absence of a character)

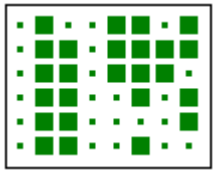
Using the Dot Meta-Character: .



- Consider: `..t..`
  - Any two characters followed by a lowercase t followed by any two characters
  - Which strings and what part would match: “the”, “and the”, “Outmost”, “Utmost”, “the utmost”, “b,o,t,h”

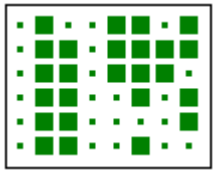
```
./a.out "..t.."
./a.out "[Tt].."
```

Start and End of String Anchors



- What if we may want to match text at the very beginning or end of a line?
- Anchor meta-characters identify where the match must be with regards to the start or end of the string of text
    - ^ (caret) indicates beginning of line
    - $ indicates end of line

Start and End of String Anchors



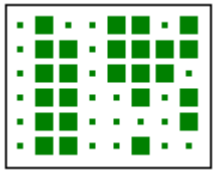
`^[Tt]he`

- Only matches “The” or “the” if it is the first text on a line

`[Tt]he$`

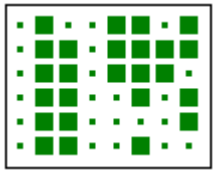
- Only matches “The” or “the” if it is the last text on a line (no other characters follow)

Finding “Words”



- `"[Tt]he"` doesn’t just match the word “the”
- To only match whole words how would we do that?
  - How would you describe the concept of a ***word*** in text?

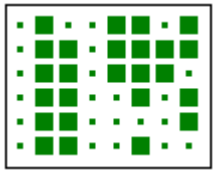
Finding “Words”



- Can’t require it be surrounded by spaces, why?
- Can’t require it have no characters next to it, why?
- These are all strings that contain the word “the”

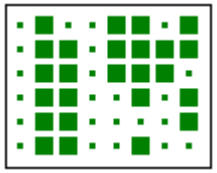
  “the,”, “the.”, “out,the”, “the”, “the end”

The Word Boundary Meta-character



- Regex defines a meta-character to represent a word boundary: **\b**
  - Does NOT represent a character
    - There may not be a character, why?
  - Identifies that there must be a word boundary found in the text at the position given in the regex
  - The \b does not stand for a word boundary character if there is one (e.g. space, comma, period)

The Word Boundary Meta-character

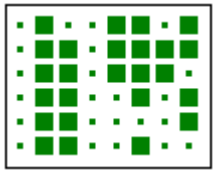


- A few examples

`[Tt]he\b`	match if letters end a word
`\b[Tt]he`	match if letters start a word
`\b[Tt]he\b`	match word “The” or “the”
`\b[Tt]he\bwork`	does NOT match “the work”

`\b[Tt]he\b work` matches “the work”

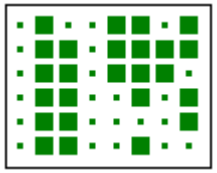
Specifying Alternate Patterns (1/2)



- If pattern could be made up of different sets of atoms use the pipe meta-character: `|`
  - OR relationship between the atoms
- `the|The` matches the same text as `[Tt]he`
  - Less efficient for searching since the regex engine searches for each separately
- Pipe is best when the atoms are truly different

  `be|am|is|are` finds some form of “be”
  - Not really, what is wrong?

Specifying Alternate Patterns (2/2)



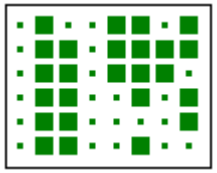
- Add word boundary to our previous example

```
\bbe|is|am|are\b
```

- Try it, what is wrong?

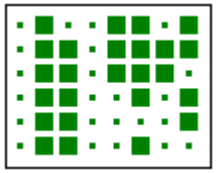
```
./a.out "\bbe|is|am|are\b"
```

Specifying Alternate Patterns (2/2)



- Precedence – every language has to deal with this concept
  - Precedence of pipe is very low
- As written, the pattern finds one of:
  1) Word boundary followed by **be**
  2) Characters **is**
  3) Characters **am**
  4) Characters **are** followed by word boundary
- Whichever pattern matches first on an input line is the one reported

Grouping with Parentheses



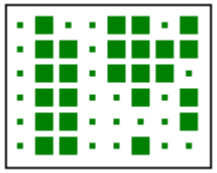
- Regex may be grouped
  - Groups define part of the regex pattern as a separate unit of atoms
  - Defined by placing pattern in parentheses
- For precedence issue we group the patterns and specify word boundary before and after

```
\b(be|am|is|are)\b
```

- Match text with word boundary followed by **be**, **am**, **is**, or **are** followed by word boundary

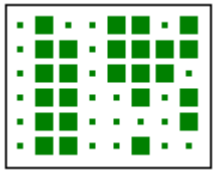
```
./a.out "\b(be|am|is|are)\b"
```

Optional Items



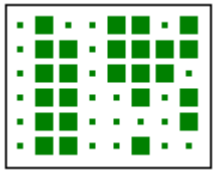
- A character, character set, or group may be made optional by use of the question mark: **?**
  - Placed after the optional item
- `[Tt]`he? makes an **e** after **Th** or **th** optional
  - Matches any **th** or **Th**
  - The **e** will be included in match if present
- `[Tt]`?he makes a **T** or **t** before **he** optional
  - Matches any **he**
  - Preceding **T** or **t** will be included if present

Optional Items



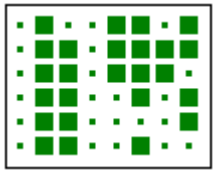
- `./a.out "[Tt]he?"`
- `./a.out "[Tt]?he"`
- `./a.out "(\bto )?\b(be|go|send)\b"`
- `./a.out "[0-9],?[0-9][0-9][0-9]"`

Repeating Items



- May want to match text with a defined length
  - Consider variable name in source code, often an alpha character followed by any number of alphanumeric characters
- Note the number of times an item may repeat using curly brackets: {*min*} or {*min,max*}
- Use asterisk for repeating 0 or more times: *
- Use plus for repeating 1 or more times: +
- The meta-character follows the item (character, character set, or group)

Limited Repetition: { }



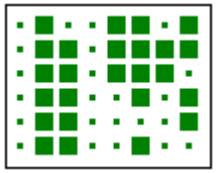
- Defines the specific number of repetitions
  - Can be a fixed number or a range
    - {*min*} item must occur exactly min times
    - {*min*,} item occurs min to infinite times
    - {*min*,*max*} item occurs between min and max times (inclusive)
    - {,*max*} item occurs 0 to max times

`[0-9]{5}` matches consecutive 5 digits

`\b[0-9]{4,5}\b` matches 4 or 5 digit “word”

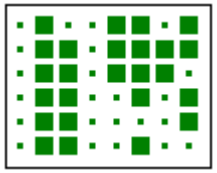
`[0-9]{3,}` matches 3 or more consecutive digits

Limited Repetition: { }



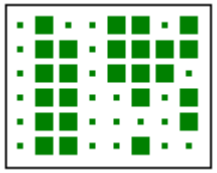
```
./a.out "\b[0-9]{1,5}\b"
./a.out "[0-9]{1,}"
./a.out "\b.{1,}[d|p|t]{2}.{1,}\b"
```

Regex Repetition is Greedy



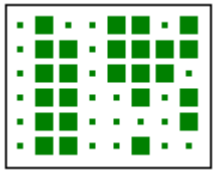
- What does greedy mean in terms of RegEx?
  - A quantifier is greedy if it matches as much of the string as possible while still allowing the whole regular expression to match
- Consider the regex: `[0-9]{4,6}`
  - If the text being evaluated is `abc1234567`
  - What will be matched? 1234, 12345, 123456?
    - Greedy match will match as much of the string as possible: `123456`

Regex Repetition is Greedy



```
/a.out "[0-9]{4,6}"
./a.out "[a-z]{2,}"
./a.out "[A-Za-z ]{2,}"
```

Repeating 0 to Infinite Times: `*`

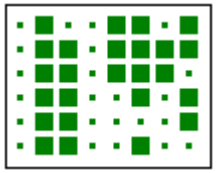


- The asterisk denotes that an item must repeat 0 or more times
  - Makes it optional, equivalent to `{0,}`
- *Beware of the asterisk*, may lead to confusion because it makes the item optional

  `A*` or `[0-9]*` or `(be)*` all match any line

```
./a.out "[24680]*"
./a.out "\b[a-z]*br[a-z]*\b"
```

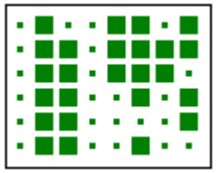
Repeating 1 to Infinite Times: +



- The plus sign denotes that an item must repeat 1 or more times
  - Makes it required, equivalent to `{1,}`

```
./a.out “[24680]+”
./a.out “\b[a-z]+br[a-z]+\b”
```

POSIX Bracket Expressions

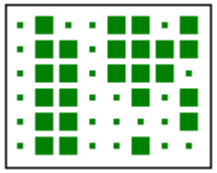


- POSIX includes a set of predefined character classes that are available via “bracket expressions”
  - These may only be used within regex character sets; square brackets `[ ]`
- The bracket expression syntax is:

  `[:class_name:]`

- `[[:alnum:]]` matches an alphanumeric character
  - Equivalent to `[A-Za-z0-9]` in ASCII

POSIX Bracket Expressions

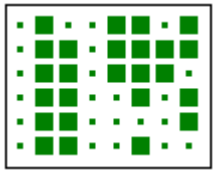


- May combine with other characters and bracket expressions in the regex character set
- `[[:upper:]0-9[:space:]]` matches any uppercase character, digit, or whitespace

```
./a.out "[[:digit:]]+"

./a.out "\b[[:alpha:]_][[:alnum:]_]*\b"
```

Regex: A Powerful Pattern Matching Language

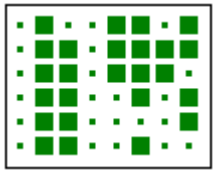


- There are a lot more capabilities provided by regex engines
  - The PDF on Perusall covers most of them in detail if you are curious
  - You will not need regex features, beyond the ones we covered, for your lexical analyzer (scanner)
- You will spend time with regular expressions during next week's Flex lab, building a basic calculator

Maybe You'll Save a Life!

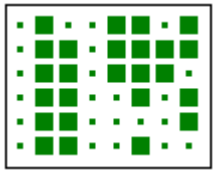


https://www.xkcd.com/208/



Emulate a Lexical Analyzer Using Some B-Minor Code

Homework



- Sunday (9/27)
  - Journal entry for week 3
    - **2-1 Rule** (minimum 2 paragraphs, 1 quiz question)
- Monday (9/28):
  - Lab 1 Due: C Coding (Team and Individual)
- Friday (10/2):
  - Compilers book Chapter 4
  - LLM: The productivity paradox…

